

FORMATION COMPLETE — Django ORM Expert + Big Data + Power BI

PARTIE 1 — Django ORM Expert (Optimisation & Requêtes Complexes)

Grok AI

December 10, 2025

Contents

1	Introduction	2
1.1	Prérequis	2
2	Module 1 — Anatomie profonde du QuerySet	3
2.1	Explication pointue	3
2.2	Exemples	3
2.2.1	Exemple Basique: Lazy Evaluation	3
2.2.2	Exemple Avancé: Analyse SQL	3
2.2.3	Exemple Entreprise Big Data: Netflix	3
3	Module 2 — F Expressions (niveau expert)	4
3.1	Explication pointue	4
3.2	Exemples	4
3.2.1	Exemple Basique: Update Atomique	4
3.2.2	Exemple Avancé: Avec Annotations	4
3.2.3	Exemple Entreprise Big Data: Google Ads	4
4	Module 3 — Q Objects (conditions complexes)	5
4.1	Explication pointue	5
4.2	Exemples	5
4.2.1	Exemple Basique: OR Condition	5
4.2.2	Exemple Avancé: Filtres Dynamiques	5
4.2.3	Exemple Entreprise Big Data: Amazon Search	5
5	Module 4 — Subquery & OuterRef	6
5.1	Explication pointue	6
5.2	Exemples	6
5.2.1	Exemple Basique: Subquery Simple	6
5.2.2	Exemple Avancé: Avec Exists	6
5.2.3	Exemple Entreprise Big Data: Netflix Recommendations	6
6	Module 5 — Window Functions	7
6.1	Explication pointue	7
6.2	Exemples	7
6.2.1	Exemple Basique: RowNumber	7
6.2.2	Exemple Avancé: Sum Over Partition	7

6.2.3	Exemple Entreprise Big Data: Google Analytics	7
7	Module 6 — Expressions SQL avancées	8
7.1	Explication pointue	8
7.2	Exemples	8
7.2.1	Exemple Basique: Cast	8
7.2.2	Exemple Avancé: Func Custom	8
7.2.3	Exemple Entreprise Big Data: Amazon Warehouse	8
8	Module 7 — Aggregations avancées	9
8.1	Explication pointue	9
8.2	Exemples	9
8.2.1	Exemple Basique: Aggregate	9
8.2.2	Exemple Avancé: Annotate avec Group By	9
8.2.3	Exemple Entreprise Big Data: Facebook Metrics	9
9	Module 8 — Prefetch personnalisé	10
9.1	Explication pointue	10
9.2	Exemples	10
9.2.1	Exemple Basique: Prefetch _{related}	10
9.2.2	Exemple Avancé: Prefetch Personnalisé	10
9.2.3	Exemple Entreprise Big Data: LinkedIn Connections	10
10	Module 9 — CTE + Raw SQL	11
10.1	Explication pointue	11
10.2	Exemples	11
10.2.1	Exemple Basique: Raw SQL	11
10.2.2	Exemple Avancé: CTE	11
10.2.3	Exemple Entreprise Big Data: Uber Analytics	11
11	Module 10 — Indexation avancée Big Data	12
11.1	Explication pointue	12
11.2	Exemples	12
11.2.1	Exemple Basique: Ajout Index	12
11.2.2	Exemple Avancé: Index GIN pour JSONB	12
11.2.3	Exemple Entreprise Big Data: Elasticsearch-like chez Twitter	12

1 Introduction

Cette formation complète est conçue pour vous transformer en expert du Django ORM, en mettant l'accent sur l'optimisation et les requêtes complexes. Nous aborderons des concepts avancés avec des explications détaillées, des exemples concrets, et des cas d'usage inspirés d'entreprises Big Data comme Google, Amazon ou Netflix, où la gestion de grandes quantités de données est critique. Chaque module inclut des explications pointues, du code Python/Django, et des exemples d'entreprise.

Nous utilisons Django 5.x pour les exemples. Assurez-vous d'avoir un environnement Django installé avec une base de données (PostgreSQL recommandé pour les features avancées).

1.1 Prérequis

- Connaissances de base en Python et Django. - Compréhension des modèles Django et des QuerySets basiques. - Accès à une base de données relationnelle.

2 Module 1 — Anatomie profonde du QuerySet

2.1 Explication pointue

Le QuerySet est au cœur du Django ORM. Il s'agit d'une collection lazy (paresseuse) d'objets qui représente une requête SQL non encore exécutée. L'évaluation lazy permet d'optimiser les performances en retardant l'exécution jusqu'au moment nécessaire (e.g., itération, slicing).

- **Lazy Evaluation:** Le QuerySet n'exécute pas la requête SQL tant qu'il n'est pas évalué (e.g., via `list()`, `for` loop, ou `len()`). - **SQL Compiler:** Django compile la requête en SQL via le backend de la base de données. Utilisez `str(queryset.query)` pour voir le SQL généré. - **Analyse SQL:** Pour debugger, activez `connection.queries` qui logue toutes les requêtes SQL exécutées. - **Connection.queries:** Une liste de dictionnaires contenant les requêtes SQL et leur temps d'exécution, utile pour profiler en environnement Big Data.

Dans un contexte Big Data, comme chez Amazon avec des milliards de transactions, comprendre l'anatomie aide à éviter les N+1 queries qui scalent mal.

2.2 Exemples

2.2.1 Exemple Basique: Lazy Evaluation

```
1 from django.db import models
2 from myapp.models import Product # Assume Product model exists
3
4 qs = Product.objects.all() # No SQL executed yet
5 print(qs) # Triggers evaluation: SELECT * FROM product
```

2.2.2 Exemple Avancé: Analyse SQL

```
1 from django.db import connection, reset_queries
2
3 reset_queries() # Clear previous queries
4 qs = Product.objects.filter(price__gt=100)
5 list(qs) # Evaluate
6
7 print(connection.queries) # [{'sql': 'SELECT ...', 'time': '0.001'}]
```

2.2.3 Exemple Entreprise Big Data: Netflix

Chez Netflix, pour analyser les vues d'utilisateurs (milliards de rows), on utilise lazy QuerySets pour chain des filters sans exécution prématurée:

```
1 from myapp.models import UserView
2
3 qs = UserView.objects.filter(user_id=12345) # Lazy
4 qs = qs.filter(view_date__year=2025) # Still lazy, chains filters
5 total_views = qs.count() # Now executes: SELECT COUNT(*) ...
```

Cela optimise en compilant une seule requête SQL optimisée.

3 Module 2 — F Expressions (niveau expert)

3.1 Explication pointue

Les F Expressions permettent de référencer des colonnes de la base de données dans les updates ou filters, évitant de charger les données en mémoire. Elles sont essentielles pour les mises à jour atomiques, éliminant les race conditions dans des environnements concurrentiels Big Data.

- **Mise à jour atomique:** Update directement en DB sans read-modify-write. - **Élimination des race conditions:** Dans un système comme un e-commerce (Amazon), où plusieurs users incrémentent un compteur stock simultanément.

F() wrappe un champ modèle, permettant des opérations arithmétiques ou logiques en SQL.

3.2 Exemples

3.2.1 Exemple Basique: Update Atomique

```
1 from django.db.models import F
2 from myapp.models import Inventory
3
4 Inventory.objects.filter(product_id=1).update(stock=F('stock') - 1)
5 # SQL: UPDATE inventory SET stock = stock - 1 WHERE ...
```

3.2.2 Exemple Avancé: Avec Annotations

```
1 qs = Inventory.objects.annotate(new_stock=F('stock') * 1.1)
2 # Puis update ou filter sur new_stock
```

3.2.3 Exemple Entreprise Big Data: Google Ads

Dans un système comme Google Ads, pour updater des clics en temps réel sans race conditions:

```
1 from myapp.models import AdCampaign
2
3 AdCampaign.objects.filter(id=campaign_id).update(clicks=F('clicks') + 1)
4 # Atomique, safe pour des millions de requêtes concurrentes.
```

Cela évite des locks manuels et scale horizontalement.

4 Module 3 — Q Objects (conditions complexes)

4.1 Explication pointue

Les Q Objects encapsulent des conditions de filtre complexes avec des opérateurs logiques (AND, OR, NOT, XOR). Ils sont dynamiques, idéaux pour des recherches avancées dans Big Data, comme des moteurs de recherche internes (e.g., Elasticsearch-like mais en SQL).

- **AND/OR/XOR/NOT:** | pour OR, pour AND, pour NOT. - **Filtres dynamiques:** Construire des queries basées sur input user.

Dans des entreprises comme Facebook, pour filtrer des posts basés sur multiples critères user-defined.

4.2 Exemples

4.2.1 Exemple Basique: OR Condition

```
1 from django.db.models import Q
2 from myapp.models import Employee
3
4 qs = Employee.objects.filter(Q(salary__gt=50000) | Q(years_experience__gt=5))
```

4.2.2 Exemple Avancé: Filtres Dynamiques

```
1 conditions = Q()
2 if user_input['high_salary']:
3     conditions |= Q(salary__gt=100000)
4 if user_input['senior']:
5     conditions &= Q(years_experience__gt=10)
6 qs = Employee.objects.filter(conditions)
```

4.2.3 Exemple Entreprise Big Data: Amazon Search

Pour un moteur de recherche produit:

```
1 search_terms = ['laptop', 'gaming']
2 q_objects = Q()
3 for term in search_terms:
4     q_objects |= Q(name__icontains=term) | Q(description__icontains=term)
5 qs = Product.objects.filter(q_objects & ~Q(stock=0)) # Exclude out-of-stock
```

Cela gère des queries complexes sur des catalogues de millions de produits.

5 Module 4 — Subquery & OuterRef

5.1 Explication pointue

Les Subqueries permettent d'imbriquer des queries, et OuterRef référence des champs de la query externe dans la subquery. Idéal pour des sous-requêtes corrélées, comme récupérer le dernier élément par groupe sans GROUP BY complexe.

- **Sous-requêtes corrélées:** Dépendent de la row externe. - **Récupération du dernier élément par groupe:** E.g., dernier achat par user.

Dans Big Data, comme chez Uber pour tracker les dernières positions de drivers.

5.2 Exemples

5.2.1 Exemple Basique: Subquery Simple

```
1 from django.db.models import Subquery, OuterRef
2 from myapp.models import Order, Customer
3
4 sub_qs = Order.objects.filter(customer=OuterRef('pk')).order_by('-date')[:1]
5 qs = Customer.objects.annotate(last_order_date=Subquery(sub_qs.values('date')))
```

5.2.2 Exemple Avancé: Avec Exists

```
1 from django.db.models import Exists
2
3 sub_qs = Order.objects.filter(customer=OuterRef('pk'), amount__gt=1000)
4 qs = Customer.objects.filter(Exists(sub_qs))
```

5.2.3 Exemple Entreprise Big Data: Netflix Recommendations

Pour annoter le dernier visionnage par user:

```
1 from myapp.models import User, ViewHistory
2
3 sub_qs = ViewHistory.objects.filter(user=OuterRef('pk')).order_by('-timestamp')[:1]
4 users = User.objects.annotate(last_view=Subquery(sub_qs.values('movie_id'))
5 # Puis utiliser pour recommandations bas es sur Big Data analytics.
```

Optimise sans joins massifs sur des TB de données.

6 Module 5 — Window Functions

6.1 Explication pointue

Les Window Functions (disponibles via PostgreSQL/MySQL 8+) permettent des calculs sur des partitions de données sans grouper, comme Rank, Sum over partition.

- **Rank, DenseRank, RowNumber**: Pour classements. - **Sum over partition**: Agrégats glissants. Essentielles pour analytics Big Data, comme ranking de ventes chez Walmart.

6.2 Exemples

6.2.1 Exemple Basique: RowNumber

```
1 from django.db.models import Window
2 from django.db.models.functions import RowNumber
3 from myapp.models import Sale
4
5 qs = Sale.objects.annotate(row_num=Window(expression=RowNumber(), order_by='-amount'))
```

6.2.2 Exemple Avancé: Sum Over Partition

```
1 from django.db.models.functions import Sum
2
3 qs = Sale.objects.annotate(total_by_region=Window(
4     expression=Sum('amount'),
5     partition_by='region',
6     order_by='-date'
7 ))
```

6.2.3 Exemple Entreprise Big Data: Google Analytics

Pour ranking de pages vues par session:

```
1 from myapp.models import PageView
2
3 qs = PageView.objects.annotate(
4     rank=Window(
5         expression=Rank(),
6         partition_by='session_id',
7         order_by='-views'
8     )
9 )
10 # Utile pour Big Data processing de logs web.
```

7 Module 6 — Expressions SQL avancées

7.1 Explication pointue

Ces outils permettent d'étendre l'ORM avec du SQL custom: ExpressionWrapper pour wrap des expressions, Func pour fonctions SQL, Cast pour conversions de types.

- **ExpressionWrapper**: Combine expressions avec `output_field`. – **Func** : *CustomSQLfunctions*. – **Cast** : *Changetype*(e.g., *stringtoint*).

Dans Big Data, pour des calculs complexes sans ETL externe (e.g., chez Hadoop-integrated systems).

7.2 Exemples

7.2.1 Exemple Basique: Cast

```
1 from django.db.models.functions import Cast
2 from django.db.models import IntegerField
3
4 qs = Product.objects.annotate(price_int=Cast('price', IntegerField()))
```

7.2.2 Exemple Avancé: Func Custom

```
1 from django.db.models import Func
2
3 class JsonExtract(Func):
4     function = 'jsonb_extract_path_text'
5     template = "%(function)s(%(expressions)s, 'key')"
6
7 qs = Data.objects.annotate(value=JsonExtract('json_field'))
```

7.2.3 Exemple Entreprise Big Data: Amazon Warehouse

Pour wrapper des calculs de distance (GIS-like):

```
1 from django.db.models import ExpressionWrapper, F, FloatField
2 from django.db.models.functions import Power, Sqrt
3
4 qs = Warehouse.objects.annotate(
5     distance=ExpressionWrapper(
6         Sqrt(Power(F('x') - user_x, 2) + Power(F('y') - user_y, 2)),
7         output_field=FloatField()
8     )
9 )
10 # Optimise routing en Big Data logistics.
```

8 Module 7 — Aggregations avancées

8.1 Explication pointue

Aggregations groupent des données: Annotate ajoute par row, Aggregate par query. Group By interne via values().

- **Group By interne:** Utiliser values() avant annotate. - **Annotate vs Aggregate:** Per-row vs global. Critique pour reporting Big Data, comme sums de revenus chez Microsoft.

8.2 Exemples

8.2.1 Exemple Basique: Aggregate

```
1 from django.db.models import Sum
2
3 total = Order.objects.aggregate(total_sales=Sum('amount'))
```

8.2.2 Exemple Avancé: Annotate avec Group By

```
1 qs = Order.objects.values('customer').annotate(total=Sum('amount')).order_by('-total')
```

8.2.3 Exemple Entreprise Big Data: Facebook Metrics

Pour agrégats par user group:

```
1 from myapp.models import Post
2
3 qs = Post.objects.values('author__country').annotate(
4     avg_likes=Avg('likes'),
5     total_posts=Count('id')
6 )
7 # Big Data insights sur engagement global.
```

9 Module 8 — Prefetch personnalisé

9.1 Explication pointue

Prefetch_{related} optimise les relations many-to-many/one-to-many en batchant les queries. Personnalisé via Prefetch() pour

- **QuerySets optimisés**: Réduit N+1 queries. - **Préchargement conditionnel**: Filter sur related objects.

Dans Big Data, comme chez Twitter pour loader timelines efficiently.

9.2 Exemples

9.2.1 Exemple Basique: Prefetch_{related}

```
1 qs = Author.objects.prefetch_related('books')
```

9.2.2 Exemple Avancé: Prefetch Personnalisé

```
1 from django.db.models import Prefetch
2
3 books_qs = Book.objects.filter(published__year=2025)
4 qs = Author.objects.prefetch_related(Prefetch('books', queryset=books_qs, to_attr='
    recent_books'))
```

9.2.3 Exemple Entreprise Big Data: LinkedIn Connections

Pour prefetch connections actives:

```
1 active_connections = Connection.objects.filter(active=True)
2 qs = User.objects.prefetch_related(Prefetch('connections', queryset=active_connections,
    to_attr='active_conns'))
3 # Scale pour r seaux sociaux Big Data.
```

10 Module 9 — CTE + Raw SQL

10.1 Explication pointue

CTE (Common Table Expressions) via WITH pour queries temporaires. Raw SQL pour mixer avec ORM quand nécessaire.

- **Requêtes WITH**: Définit des subqueries réutilisables. - **RawQuerySet**: Exécute SQL raw mais mappe à models.

Utile pour queries complexes en Big Data non supportées par ORM pur (e.g., recursive queries).

10.2 Exemples

10.2.1 Exemple Basique: Raw SQL

```
1 qs = Product.objects.raw('SELECT * FROM product WHERE price > %s', [100])
```

10.2.2 Exemple Avancé: CTE

```
1 from django.db.models import RawSQL
2
3 # CTE example (PostgreSQL specific)
4 raw_cte = RawSQL("""
5 WITH ranked AS (
6     SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY sales DESC) as rn
7     FROM product
8 )
9 SELECT * FROM ranked WHERE rn <= 3
10 """, [])
11 qs = Product.objects.annotate(rn=raw_cte).filter(rn__lte=3)
```

10.2.3 Exemple Entreprise Big Data: Uber Analytics

Pour CTE sur trajets:

```
1 # RawQuerySet pour CTE recursive (e.g., paths)
2 qs = Trip.objects.raw("""
3 WITH RECURSIVE path AS (
4     SELECT id, start, end FROM trip WHERE start = %s
5     UNION
6     SELECT t.id, p.start, t.end FROM path p JOIN trip t ON p.end = t.start
7 )
8 SELECT * FROM path
9 """, ['CityA'])
```

Gère graphes Big Data.

11 Module 10 — Indexation avancée Big Data

11.1 Explication pointue

Indexation optimise les queries sur grandes tables. Django supporte via migrations: B-Tree (default), Hash, GIN/GIST pour arrays/JSON, BRIN pour sorted data, JSONB pour JSON.

- **B-Tree**: Pour equality/range. - **Hash**: Equality only. - **GIN/GIST**: Full-text, arrays, spatial. - **BRIN**: Block Range pour grandes tables triées. - **JSONB indexes**: Pour queries JSON en Big Data NoSQL-like.

Dans entreprises comme MongoDB-integrated, mais en SQL pour scale.

11.2 Exemples

11.2.1 Exemple Basique: Ajout Index

Dans models.py:

```
1 class Product(models.Model):
2     name = models.CharField(max_length=100, db_index=True) # B-Tree
```

11.2.2 Exemple Avancé: Index GIN pour JSONB

```
1 from django.contrib.postgres.indexes import GinIndex
2
3 class Meta:
4     indexes = [GinIndex(fields=['data'], name='data_gin_idx')]
5 # Pour data = JSONField()
```

11.2.3 Exemple Entreprise Big Data: Elasticsearch-like chez Twitter

Pour index sur tweets JSON:

```
1 from django.contrib.postgres.indexes import BrinIndex, GistIndex
2
3 class Tweet(models.Model):
4     content = models.TextField()
5     metadata = models.JSONField()
6
7     class Meta:
8         indexes = [
9             BrinIndex(fields=['created_at']), # Pour time-series Big Data
10            GinIndex(fields=['metadata'])
11        ]
12 # Queries: Tweet.objects.filter(metadata__key='value')
```

Optimise searches sur PB de données.